

2026 Spring

Course ID: 1142-1765

Advanced Programming



06-C++ for 'Maximum Subarray'



Advanced Programming

Python · C++ · Rust

Data Structures & Algorithms



Python



C++



Rust

Part-1

C++
Syntax

LeetCode Problem
"Maximum Subarray"

Course Activities
Discuss midterm
presentation

C++ Course

Fundamentals

Input / Output / for / if / vector / string

I/O

for

if

vector

string



Input & Output

```
#include <iostream>
```

Header for I/O stream

```
cout <<
```

Output to console (print)

```
cin >>
```

Read input from user

```
endl / \n
```

Newline / flush buffer



```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter age: ";
    cin >> age;
    cout << "Your age: " << age << endl;
    return 0;
}
```




for Loop

Syntax:

```
for (init; condition; update) { statement(s); }
```

Init

```
int i = 0
```

Runs once at start

Condition

```
i < 10
```

Checked each iteration

Update

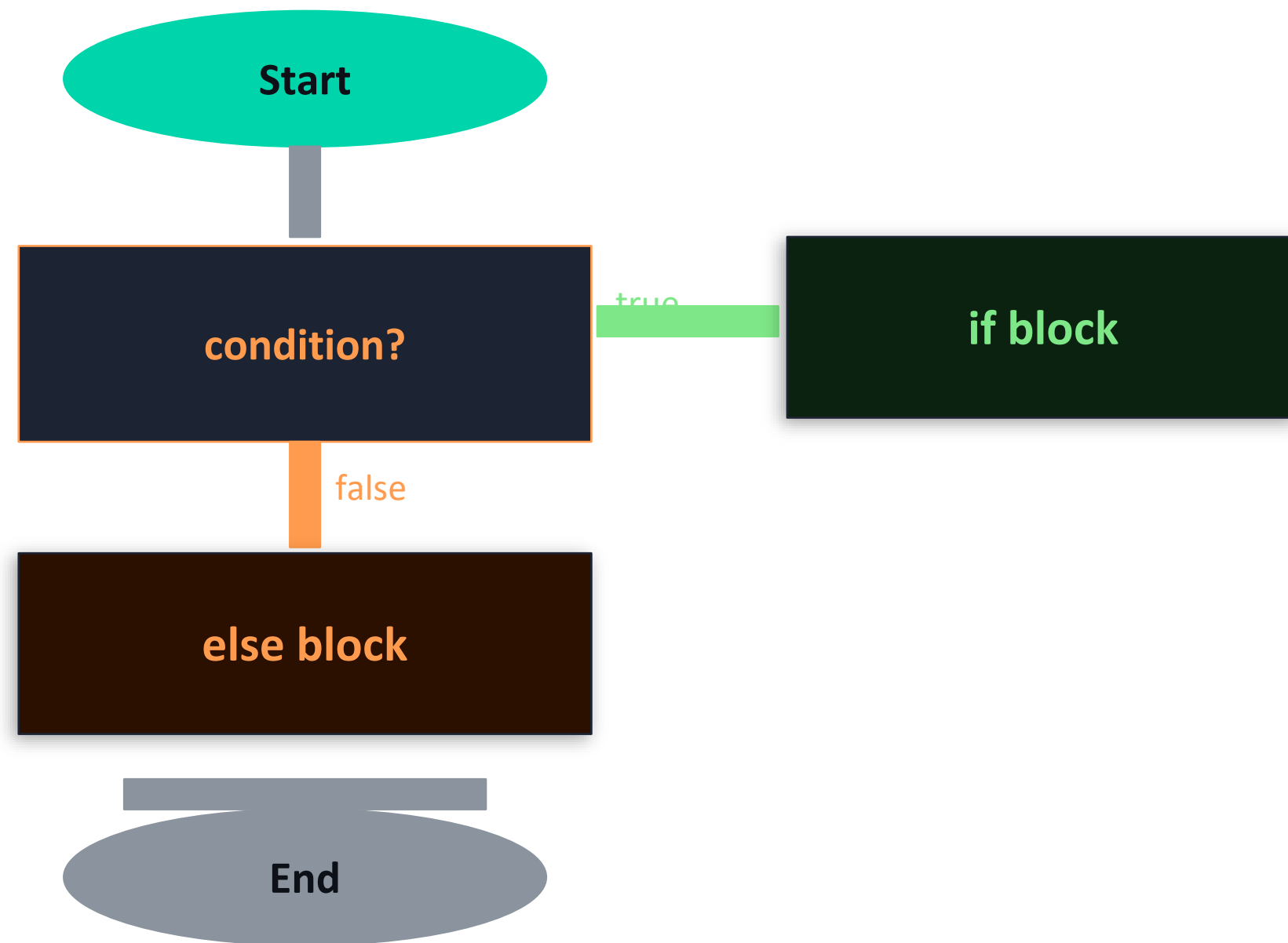
```
i++
```

Runs after each loop

```
// Classic for loop
for (int i = 0; i < 5; i++) {
    cout << i << " ";
}
// Output: 0 1 2 3 4
```

```
// Range-based for loop
vector<int> nums = {1,2,3};
for (int n : nums) {
    cout << n << " ";
}
```

⚡ if / else Statement



```
int score = 85;

if (score >= 90) {
    cout << "A grade";
} else if (score >= 70) {
    cout << "B grade";
} else {
    cout << "Need to study";
}
```

Comparison Operators:

==

equal

!=

not equal

>

greater

<

less

>=

≥

<=

≤



std::vector

Dynamic array in memory:



push_back(x)

Add element to end

pop_back()

Remove last element

size()

Return element count

at(i) / [i]

Access element at i

clear()

Remove all elements

empty()

Check if empty



```
#include <vector>
using namespace std;

vector<int> v = {1, 2, 3};
v.push_back(4);
cout << v.size(); // 4
for (int x : v)
    cout << x << " ";
// 1 2 3 4
```




std::string

"Hello" is stored as:



length()/size()

Number of characters

substr(pos, n)

Extract substring

find(str)

Find position of substr

+ / append()

Concatenate strings

at(i) / [i]

Access character at i

compare(s)

Compare two strings



```
#include <string>
using namespace std;

string s = "Hello";
string t = "World";

cout << s + " " + t;
// Hello World

cout << s.length();
// 5
```



Summary

I/O

cin / cout

Read input and print output to the terminal

for

Classic & Range-based

Repeat code a fixed number of times or over a list

if

if / else if / else

Conditionally execute code based on true/false

vector

push_back, size, at()

Dynamic array that can grow and shrink at runtime

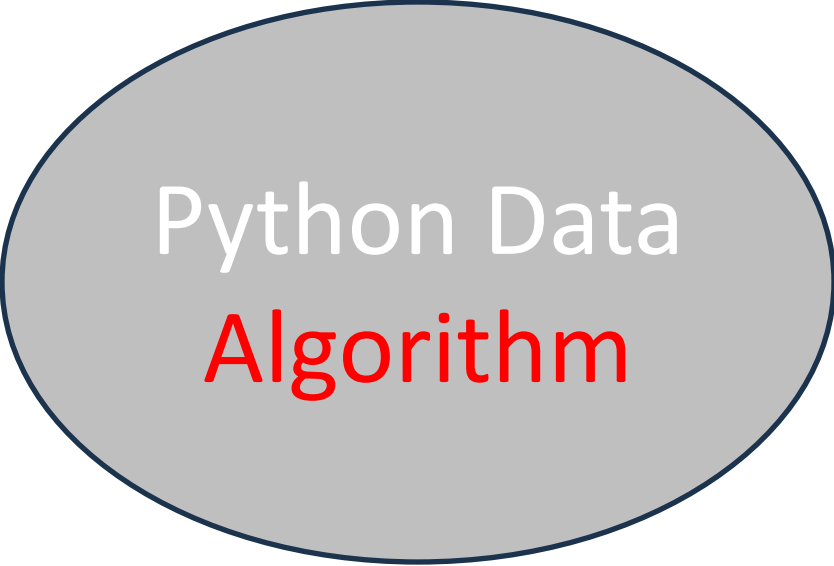
string

length, substr, find, +

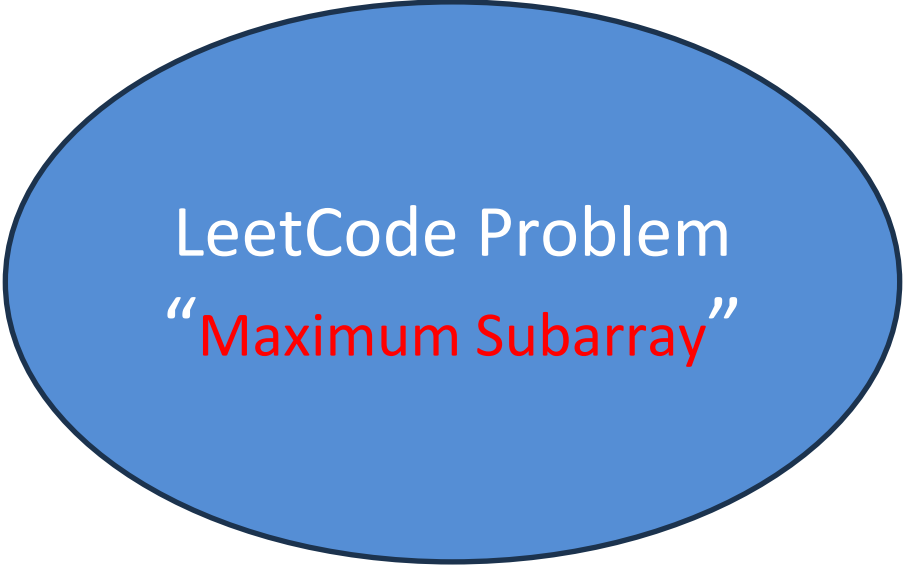
Sequence of characters with powerful text methods

Keep coding! Practice makes perfect.

Part-2



Python Data
Algorithm



LeetCode Problem
"Maximum Subarray"



Course Activities

LeetCode Practice

Master Programming Skills with Python · C++ · Rust

2500+

Problems

50+

Companies

10M+

Users

Difficulty Tiers



Easy



Medium



Hard

01. Two Sum

 PUAP01_Python_quick_tutorial.ipynb

檔案 編輯 檢視畫面 插入 執行階段 工具 說明

Q 指令 + 程式碼 + 文字 | 全部執行 複製到雲端硬碟 連線



```
[ ] 1 instr = input('Please input your score: ')
2 score = int(instr) #Integer
3 print(score) # print the value of the variable
4 print(type(score)) # print the type of the variable
```

Please input your score: 88
88
<class 'int'>

Arithmetic Operators and Expressions Arithmetic Operators

```
+ - * / #Add, subtract, multiply and divide
** #power
// #business
% #remainder
```

```
[ ] 1 print(1 + 2)
2 print(3 - 4)
```

變數 終端機

<https://leetcode.com/problems/two-sum/description/>
<https://leetcode.cn/problems/two-sum/description/>

66. Plus One

Problem List

Submit

Register

Description

Editorial

Solutions

Submissions

66. Plus One

Easy Topics Companies

You are given a **large integer** represented as an integer array `digits`, where each `digits[i]` is the i^{th} digit of the integer. The digits are ordered from most significant to least significant in left-to-right order. The large integer does not contain any leading 0's.

Increment the large integer by one and return *the resulting array of digits*.

Example 1:

Input: `digits = [1,2,3]`

Output: `[1,2,4]`

Explanation: The array represents the integer 123. Incrementing by one gives $123 + 1 = 124$. Thus, the result should be `[1,2,4]`.

Example 2:

Input: `digits = [4,3,2,1]`

Output: `[4,3,2,2]`

Explanation: The array represents the integer 4321. Incrementing by one gives $4321 + 1 = 4322$. Thus, the result should be `[4,3,2,2]`.

</> Code

Python3 • Auto

```
1 class Solution:
2     def plusOne(self, digits: List[int]) -> List[int]:
3
```

Saved

You need to [log in / sign up](#) to run or submit

Testcase

Test Result

You must run your code first

<https://leetcode.com/problems/plus-one/description/>
<https://leetcode.cn/problems/plus-one/description/>

20. Valid Parentheses

Description

Editorial

Solutions

Submissions

20. Valid Parentheses

Easy

Topics

Companies

Hint

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: `s = "()"`

Output: `true`

</> Code

Python3 • Auto

```
1 class Solution:
2     def isValid(self, s: str) -> bool:
3
```

Saved

You need to [log in / sign up](#) to run or submit

☒ Testcase

> Test Result

You must run your code first

<https://leetcode.com/problems/valid-parentheses/>
<https://leetcode.cn/problems/valid-parentheses/>

704. Binary Search

Problem List

<

>

Submit

Register or Log in

Premium

Description

Editorial

Solutions

Submissions

704. Binary Search

EasyTopicsCompanies

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return `-1`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [-1,0,3,5,9,12], target = 9`

Output: `4`

Explanation: 9 exists in `nums` and its index is 4

Example 2:

Input: `nums = [-1,0,3,5,9,12], target = 2`

Output: `-1`

Explanation: 2 does not exist in `nums` so return -1

</> Code

Python3 • Auto

```
1 class Solution:
2     def search(self, nums: List[int], target: int) -> int:
3 
```

SavedLn 3, Col 9

You need to [log in / sign up](#) to run or submit

Testcase

>

Test Result

You must run your code first

<https://leetcode.com/problems/binary-search/>
<https://leetcode.cn/problems/binary-search/>

53. Maximum Subarray

Problem List

Submit

Register or Log in

Premium

Description

Editorial

Solutions

Submissions

53. Maximum Subarray

Medium Topics Companies

Given an integer array `nums`, find the **subarray** with the largest sum, and return *its sum*.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

37.8K 519 293 Online

Code

Python3 • Auto

```
1 class Solution:
2     def maxSubArray(self, nums: List[int]) -> int:
3
```

Saved Ln 1, Col 1

You need to [log in / sign up](#) to run or submit

Testcase

Test Result

You must run your code first

<https://leetcode.com/problems/maximum-subarray/description/>

<https://leetcode.cn/problems/maximum-subarray/description/>

Kadane's Algorithm



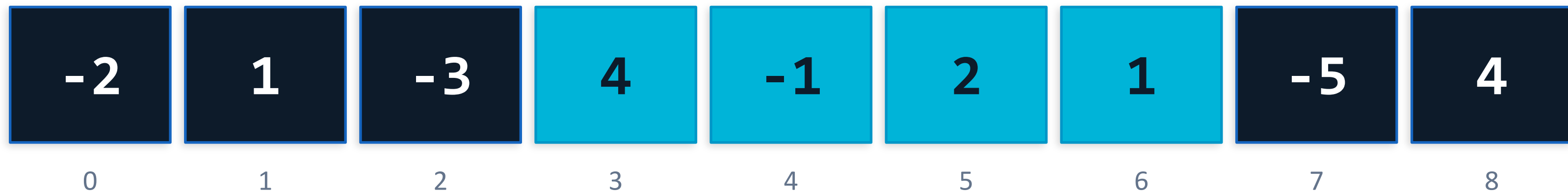
A Linear-Time Solution to a Classic Problem

Topics: Problem Definition · Brute Force · Kadane's Algorithm · Trace · Complexity

Problem Definition

What is the Maximum Subarray Problem?

Given an integer array, find the contiguous subarray with the largest sum and return that sum.



▲ Max Subarray: [4, -1, 2, 1] → Sum = 6

LeetCode 53 — Maximum Subarray

- **Input:** An integer array `nums`

- **Output:**

The largest sum of any contiguous subarray

- **Constraint:**

Array may contain negative numbers; at least one element

Brute Force Approach

$O(n^2)$ — Check every subarray

Python

```
def max_subarray_brute(nums):  
    n = len(nums)  
    max_sum = float('-inf')  
  
    for i in range(n):  
        curr = 0  
        for j in range(i, n):  
            curr += nums[j]  
            max_sum = max(max_sum, curr)  
    return max_sum
```

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Approach

Nested loops — every (i, j) pair

Limitation

Too slow for large n

Kadane's Algorithm — Core Idea

Dynamic Programming in linear time

Key Insight:

At each position, decide: extend the current subarray or start fresh from here.

1

Extend or Restart

```
current = max(nums[i],  
              current + nums[i])
```

If adding the current number helps, extend. Otherwise start fresh.

2

Track Maximum

```
max_sum = max(max_sum,  
              current)
```

After each step, update the global best seen so far.

3

Linear Scan

```
for num in nums:  
    ...
```

Process every element exactly once — $O(n)$ total.

Kadane's Algorithm — Implementation

Clean $O(n)$ Python solution

Python

```
def max_subarray(nums):  
    # Init with first element  
    current = max_sum = nums[0]  
  
    for num in nums[1:]:  
        # Extend or restart  
        current = max(num, current + num)  
  
        # Update global max  
        max_sum = max(max_sum, current)  
  
    return max_sum
```

Recurrence

$\text{current}[i] = \max(\text{nums}[i], \text{cur}[i-1] + \text{nums}[i])$

Either start new at i , or extend the previous subarray.

Global Maximum

$\text{max_sum} = \max(\text{max_sum}, \text{current}[i])$

Update the answer whenever the running sum is the new best.

Complexity

Time: $O(n)$ Space: $O(1)$

One pass through the array, constant extra memory.

Step-by-Step Trace

Input: [-2, 1, -3, 4, -1, 2, 1, -5, 4]

i	nums[i]	current = max(num, cur+num)	max_sum	Step i=6 max_sum peaks at 6 Subarray: [4, -1, 2, 1]
0	-2	$\max(-2, -) = -2$	-2	
1	1	$\max(1, -2 + (1)) = \max(1, -1) = 1$	1	
2	-3	$\max(-3, 1 + (-3)) = \max(-3, -2) = -2$	1	
3	4	$\max(4, -2 + (4)) = \max(4, 2) = 4$	4	
4	-1	$\max(-1, 4 + (-1)) = \max(-1, 3) = 3$	4	
5	2	$\max(2, 3 + (2)) = \max(2, 5) = 5$	5	
6	1	$\max(1, 5 + (1)) = \max(1, 6) = 6$	6	
7	-5	$\max(-5, 6 + (-5)) = \max(-5, 1) = 1$	6	
8	4	$\max(4, 1 + (4)) = \max(4, 5) = 5$	6	

Complexity & Edge Cases

Performance guarantees and tricky inputs

Complexity Comparison

Approach	Time	Space
Brute Force	$O(n^2)$	$O(1)$
Divide & Conquer	$O(n \log n)$	$O(\log n)$
Kadane's DP	$O(n)$	$O(1)$

Edge Cases to Know

All Negatives [-5, -3, -1] → -1
Single Element [42] → 42
All Positives [1, 2, 3, 4] → 10
Mixed with Zero [0, -1, 0, 2] → 2

Variant — Return Indices: Track start/end pointers when current resets or max_sum updates, to recover the actual subarray bounds, not just the sum.

01

Problem

Find the contiguous subarray with the maximum sum in an integer array.

02

Key Decision

`current = max(num, current + num)` – extend or start fresh at each step.

03

Algorithm

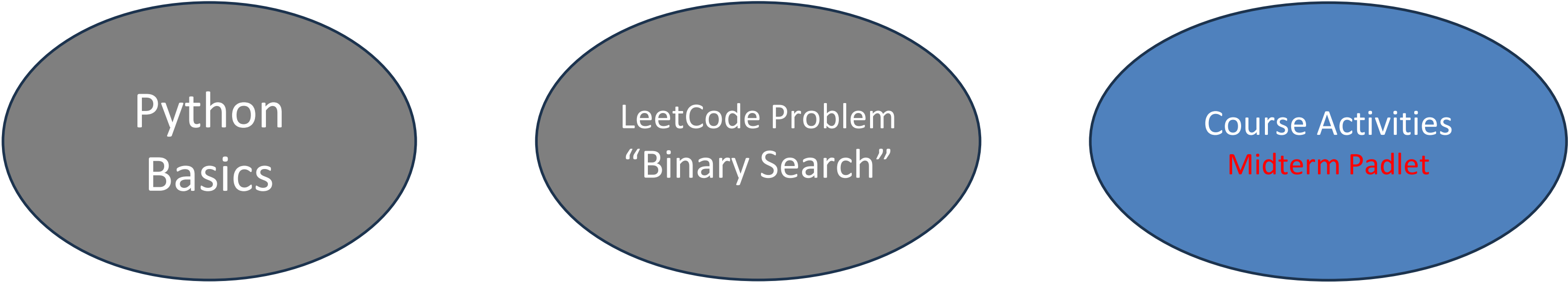
Single loop: update current sum; keep track of max_sum seen so far.

04

Complexity

Time $O(n)$ – one pass; Space $O(1)$ – no extra data structure needed.

Part-3



Python
Basics

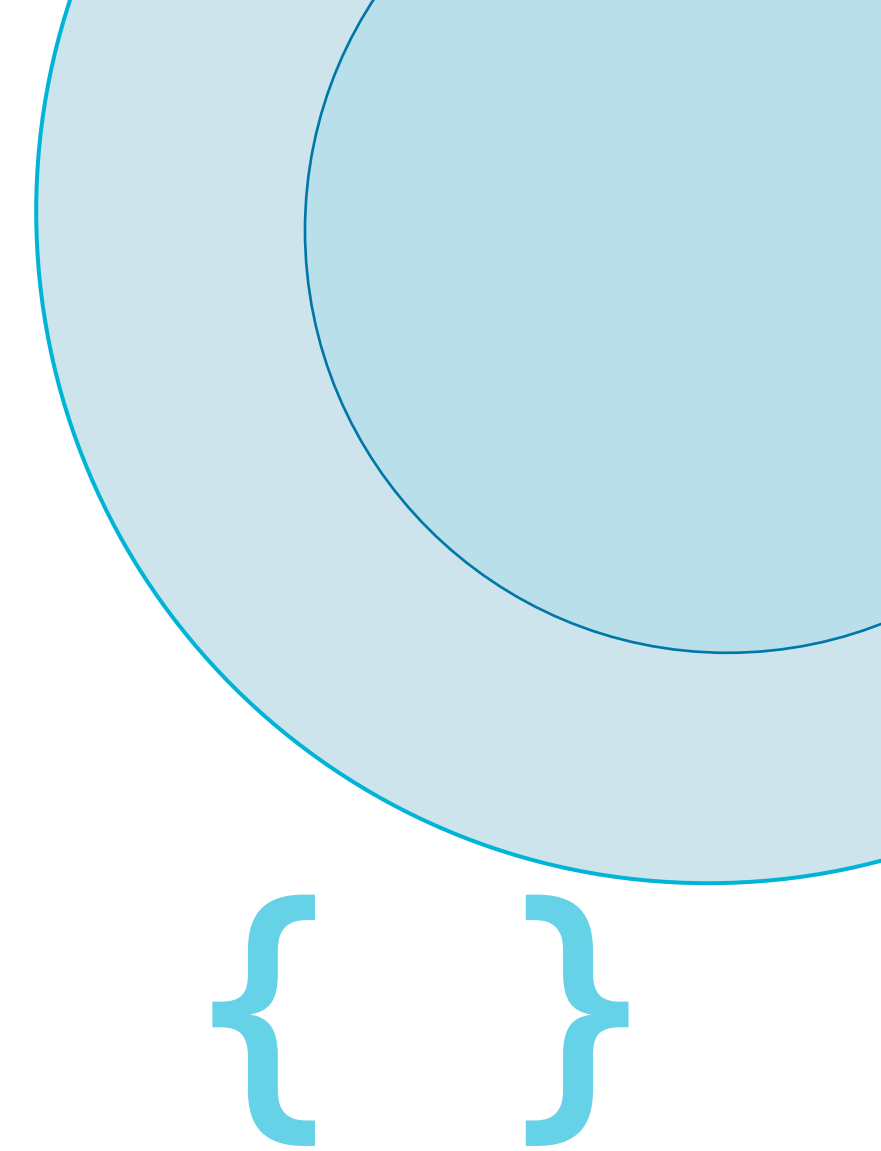
LeetCode Problem
“Binary Search”

Course Activities
Midterm Padlet

Midterm Project

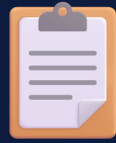
LeetCode Collaborative Learning

Mastering Algorithms through Teamwork & Documentation



Objective

Analyze, solve, and document a classic LeetCode problem as a high-performing engineering team.



Collaboration

Padlet

Real-time reporting & team discussion



Version Control

GitHub

Code management & commit history



Problem Solving

LeetCode

Solution verification & testing

Assigned Topics

03

Each group will be assigned ONE of the following problems:

#001

LeetCode 1: Two Sum

Hash Map & Array

#066

LeetCode 66: Plus One

Array Logic & Carries

#020

LeetCode 20: Valid Parentheses

Stack & LIFO

#704

LeetCode 704: Binary Search

Divide & Conquer

#053

LeetCode 53: Maximum Subarray

Dynamic Programming / Kadane's

Team Roles (5 Members)

04

1



I/O Specialist

Analyze problem constraints and edge cases

2



Algorithm Architect

Design the logic and analyze $O(n)$ complexity

3



Python Developer

Implement solution in Python + LeetCode screenshot

4



C++ Developer

Implement solution in C++ + Performance comparison

5



Team Leader

Manage GitHub Repository and README.md integration

Every member has a specific mission in the Dadlet "Shelf"

Layout

Use the **"Shelf" (Columns)** format in Padlet.

Each team member owns one column. Posts must be clearly labelled and organized for peer review.

Interaction Rules

1

Enable Comments and Reactions on every post.

2

Members must review each other's work (e.g., C++ dev commenting on Python efficiency).

3

All posts must be tagged with the author's name.

 Repository Name:

LeetCode_Project_[Group_Number]

File Structure

 **src/**

Folder containing solution.py and solution.cpp

 **README.md**

The “Final Report” summarizing all Padlet findings

 **docs/**

Screenshots of LeetCode acceptance



Technical Accuracy

Code correctness and complexity analysis

40%



Collaboration

Active discussion and peer review on Padlet

20%



Documentation

Clarity of Markdown, charts, and explanations

20%



Version Control

Proper GitHub usage and commit history

20%



Group-1

411330111 安佐駙 Tserendash.G
411330145 安佳卡 Nomunbileg Azjargal
411330200 安素恩 Lkhamsuren Ulziikhishig
411426140 庫爾特 KURT CZEDRICK SASUMAN LANDICHO
411419274 尔汀巴塔 ERDENEBAATAR AMARBAYAN
411419282 涂尔巴亞 BATJARGAL TURBAYAR

Group-2

411419313 杰 登 ROUSE JAYDEN ARIEL ANTYONNE
411419321 馬坎亞 Xolani Ignicious Makhanya
411425194 包格倫 GLENN PASCUA BAUTISTA JR.
411425217 艾德華 EDWARD ASIA CASTILLO
411426077 婕 登 MUHLE JADEN VILANE
411416959 歐匠伯 Abraham

Group-3

411412141 洪健智
411200594 黃宇碩
411211456 林靖翔
411211650 陳冠中
411120320 梁偉航
E11410470 溫依薇 WADDINGTON EVEY

Group-4

411314717 李研熙 Gloria Ananda Justine
411416878 鄭鎧文 CALVIN THEODORIC LAVENCIO
411416886 蔡美彥 MANDY JANNYCIA MAHIDIN
411416991 許原盛 Yansen Ladova
411419355 吳加寶 WILLYANTO SUDINO

Group-5

411425178 鄭永全 YOSEF CHRISTIAN
411425225 吉 曼 AHMAD SOLEHUDIN AL AZIZ
411419305 譚碧雄 CHARLES SEPTIAN PRATAMA
411419258 雨海南 VU HAI NAM
E11420600 哈里哈蘭 UDAYA SANKAR HARIHARAN

